网络规划工具 V3.0 部署指南

> 涵盖 Armbian（Linux ARM）和 Windows 两种平台的完整部署方案

文档信息

- 版本：v3.0
- 更新日期：2026-06-17
- 适用场景：新服务器迁移部署 / 系统重装后恢复

一、系统架构对比

1.1 整体架构

```
用户浏览器
    |
    ├─── 外网访问 https://<域名>/ → Cloudflare Tunnel → localhost:8000 → FastAPI 后端
    |                                                          |
    |                                                          └─── 静态文件服务（前端 dist-renderer/）
    |
    └─── 本地访问 http://localhost:8000 → FastAPI 后端（生产模式，直接服务静态文件）
```

1.2 Armbian vs Windows 关键差异

项目	Armbian（Linux ARM）	Windows
Python 环境	系统 Python 3.11+	虚拟环境 venv
进程管理	systemd 服务	Windows 服务 / 后台进程
后端启动	systemctl 管理	start-backend.bat / 后台进程
编码问题	无（天然 UTF-8）	需设置 PYTHONIOENCODING=utf-8, PYTHONUTF8=1
进程池模式	fork（默认）	spawn（Windows 必须）
隧道管理	systemd + cloudflared	Windows Service + cloudflared
Electron 桌面	不支持（无 GUI）	完整支持
文件路径	/home/xxx/...	D:\mycode\...

1.3 端口与服务对照

服务	端口	说明
FastAPI 后端	8000	生产模式同时提供 API 和前端静态文件
Vite 开发服务器	5173	仅开发时使用，生产无需
Cloudflare Tunnel	无固定端口	通过 Cloudflare 边缘节点转发

二、环境准备

2.1 基础依赖版本要求

- Python：3.11+（推荐 3.11-3.12）
- Node.js：20+（前端构建使用）
- npm：10+
- pip：最新版本

2.2 Armbian 环境准备

```
# 1. 更新系统
sudo apt update && sudo apt upgrade -y

# 2. 安装 Python 和 Node.js
sudo apt install -y python3 python3-pip python3-venv
sudo apt install -y nodejs npm

# 3. 验证版本
python3 --version  # 需 >= 3.11
node --version     # 需 >= 20

# 4. 安装 cloudflared
# 下载 ARM64 版本
wget https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-arm64
sudo dpkg -i cloudflared-linux-arm64.deb
cloudflared version

# 5. 安装 git（拉取代码）
sudo apt install -y git
```

2.3 Windows 环境准备

```
# 1. 安装 Python 3.11+
# 下载安装包 https://www.python.org/downloads/
# 安装时务必勾选 "Add Python to PATH"

# 2. 安装 Node.js 20+
# 下载安装包 https://nodejs.org/
# 安装时勾选 "Add to PATH"

# 3. 验证安装
python --version
node --version
npm --version

# 4. 安装 cloudflared
# 下载 Windows 安装包
# https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/downloads/
# 运行安装程序，默认安装在 C:\Program Files (x86)\cloudflared\

# 5. 安装 git
# https://git-scm.com/download/win
```

三、部署步骤

3.1 代码获取

```
# 两种平台通用
mkdir -p /opt/projects  # Linux
# 或
mkdir D:\mycode         # Windows

# 克隆代码（或通过 USB / scp 传输）
cd /opt/projects
git clone <仓库地址> NetworkPlanningTooV3
```

3.2 后端 Python 环境部署

Armbian（Linux ARM）

```
cd /opt/projects/NetworkPlanningTooV3/backend

# 创建虚拟环境
python3 -m venv venv
source venv/bin/activate

# 升级 pip
pip install --upgrade pip

# 安装核心依赖
pip install fastapi uvicorn[standard] python-multipart websockets pandas openpyxl numpy pydanti

# 安装地理数据处理（MapInfo TAB 支持）
pip install geopandas fiona pyproj

# 或者使用 requirements.txt 一次性安装
# 注意：pip 直接安装到系统 Python 的 site-packages，虚拟环境可独立安装
```

geopandas 安装说明：

geopandas 依赖 GDAL/Fiona，Linux 上容易安装，Windows 稍复杂。如果上述 pip 方式安装失败，按以下方式操作：

```
# Linux Armbian 备用方案
sudo apt install -y python3-numpy python3-pandas python3-shapely python3-fiona python3-geopanda
```

Windows

```
cd D:\mycode\NetworkPlanningTooV3\backend

# 创建虚拟环境
python -m venv venv

# 激活虚拟环境
.\venv\Scripts\activate

# 升级 pip
python -m pip install --upgrade pip

# 安装核心依赖（使用引号避免 PowerShell 转义问题）
$packages = @(
    "fastapi", "uvicorn[standard]", "python-multipart", "websockets",
    "pandas", "openpyxl", "numpy<2.0", "pydantic", "pydantic-settings",
    "sqlalchemy", "python-jose[cryptography]", "passlib[bcrypt]", "bcrypt",
    "python-dotenv", "cryptography", "aiofiles", "shapely"
)
$packages.ForEach({ pip install $_ })
```

Windows 上 geopandas 的特殊处理：

```
# 方法1：直接 pip（推荐先尝试）
.\venv\Scripts\python -m pip install geopandas fiona pyproj

# 方法2：如果方法1失败，使用 conda（需安装 Miniconda）
conda install -c conda-forge geopandas

# 方法3：预编译 wheel（最后手段）
# 下载地址：https://www.lfd.uci.edu/~gohlke/pythonlibs/
# 下载对应的 .whl 文件手动安装
```

Windows 额外注意：`python-calamine` 依赖

```
.\venv\Scripts\python -m pip install python-calamine
```

3.3 配置文件

复制 `backend/.env` 并根据实际环境修改关键参数：

```
# backend/.env

# 服务地址
NPT_HOST=0.0.0.0
NPT_PORT=8000

# CORS 跨域配置（根据实际域名修改）
NPT_CORS_ORIGINS=http://localhost:5173,http://127.0.0.1:5173,https://你的域名

# JWT 密钥（生产环境必须修改！）
NPT_JWT_SECRET_KEY=替换为随机生成的密钥
NPT_JWT_ACCESS_TOKEN_EXPIRE_MINUTES=1440

# 高德地图 API（可选）
NPT_AMAP_API_KEY=你的高德Key
NPT_AMAP_SECURITY_CODE=你的安全密钥
```

密钥生成建议：

```
# 在 Python 中执行生成随机密钥
python -c "import secrets; print(secrets.token_urlsafe(32))"
```

3.4 前端构建

```
cd /opt/projects/NetworkPlanningTooV3/frontend   # Linux
# 或
cd D:\mycode\NetworkPlanningTooV3\frontend       # Windows

# 安装依赖
npm install --legacy-peer-deps

# 构建生产版本
npm run build:vite

# 构建完成后，dist-renderer/ 目录即为静态文件
# ls dist-renderer/ 应包含 index.html 和 assets/ 目录
```

> **原理说明：** 前端构建后，FastAPI 后端通过 `StaticFiles` 直接服务 `dist-renderer/` 目录。因此生产环境下只需启动后端，无需启动 Vite 开发服务器。

四、服务启动与管理

4.1 Armbian（systemd 管理）

创建 systemd 服务文件

```
sudo nano /etc/systemd/system/npt-backend.service
```

```
[Unit]
Description=Network Planning Tool Backend
After=network.target

[Service]
Type=simple
User=你的用户名
WorkingDirectory=/opt/projects/NetworkPlanningTooV3/backend
ExecStart=/opt/projects/NetworkPlanningTooV3/backend/venv/bin/python main.py
Restart=always
RestartSec=5
Environment="PYTHONIOENCODING=utf-8"
Environment="PYTHONUTF8=1"
```

```
[Install]
WantedBy=multi-user.target
```

启动后端

```
sudo systemctl daemon-reload
sudo systemctl enable npt-backend
sudo systemctl start npt-backend

# 查看状态
sudo systemctl status npt-backend

# 查看日志
sudo journalctl -u npt-backend -f
```

创建 systemd 隧道服务

```
sudo nano /etc/systemd/system/npt-tunnel.service
```

```
[Unit]
Description=Cloudflare Tunnel for NPT
After=network.target

[Service]
Type=simple
User=你的用户名
ExecStart=/usr/local/bin/cloudflared tunnel run <隧道 ID 或名称>
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

```
# 启动隧道
sudo systemctl enable npt-tunnel
sudo systemctl start npt-tunnel
```

Armbian 日常操作

```
# 查看后端状态
sudo systemctl status npt-backend

# 查看后端日志（实时）
sudo journalctl -u npt-backend -f

# 重启后端
sudo systemctl restart npt-backend

# 停止后端
sudo systemctl stop npt-backend

# 停止隧道
sudo systemctl stop npt-tunnel
```

4.2 Windows

后端启动方式

方式 A — 后台进程（推荐）：

```
cd D:\mycode\NetworkPlanningTooV3\backend
```

```
# 启动（隐藏窗口）
$p = Start-Process -FilePath ".\venv\Scripts\python.exe" -ArgumentList "main.py" -WindowStyle H

# 记录 PID
$p.Id

# 停止
taskkill /PID <PID> /F
```

方式 B — 批处理脚本：

```
# 直接运行（窗口保持打开）
.\scripts\start-backend.bat
```

端口冲突处理

```
# 检测端口 8000 占用
netstat -aon | findstr ":8000" | findstr "LISTENING"

# 强制释放端口
taskkill /F /PID <占用端口的 PID>
```

Cloudflare Tunnel 服务安装（Windows Service）

注意：上述 `--token` 中的令牌仅为示例，实际安装时请替换为你自己隧道的真实令牌。

```
# 1. 以管理员身份打开 PowerShell

# 2. 安装为 Windows 服务（使用隧道 token）
# 在 Cloudflare Zero Trust 面板 → Networks → Tunnels → 点你的隧道
# → 右侧 "Copy tunnel token"
cloudflared.exe service install eyJhIjoi...

# 3. 启动服务
net start cloudflared

# 4. 查看服务状态
sc query cloudflared
```

> **注意：** `cloudflared.exe service install` 会替换已有的 cloudflared 服务。

> 每个 Windows 机器只能运行一个 cloudflared 服务连接到一个隧道。

> 如果一台机器需要服务多个域名，在 Zero Trust 面板的同一个隧道下添加多条 Public Hostname 路由即可。

卸载旧服务

```
# 以管理员身份运行
net stop cloudflared
cloudflared.exe service uninstall
```

Windows 一键启动

```
# 一键启动所有服务（后端 + 可选 Tunnel）
.\scripts\start-all.bat
```

五、Cloudflare Tunnel 配置（通用）

5.1 工作原理

```
用户 → https://你的域名 → Cloudflare 边缘节点 → Tunnel Connector（本地） → localhost:8000 → FastAPI
```

5.2 创建隧道

登录 [Cloudflare Zero Trust](https://one.dash.cloudflare.com) → **Networks → Tunnels** → **Create a tunnel**。

5.3 配置路由（Public Hostname）

在隧道编辑页面添加：

字段	值
Subdomain	你的前缀（如 win, app, apk）
Domain	你的域名（如 pengwc.asia）
Type	HTTP
URL	`localhost:8000` 或 `127.0.0.1:8000`

5.4 重要：多域名路由规则

- **同一台机器上多个域名**：在同一个隧道下添加多条 Public Hostname 即可
- **不同机器用不同域名**：需要创建不同的隧道，各机器运行各自的隧道连接
- 一个隧道可以被多台机器连接（称为"副本"），Cloudflare 会负载均衡分发请求到所有副本
- 要隔离域名到特定机器，必须使用不同的隧道

5.5 Cloudflare DNS 记录

在 Cloudflare Dashboard → **DNS** → 添加记录：

类型	名称	内容
CNAME	你的前缀	`<隧道UUID>.cfargotunnel.com`

> 配置了 tunnel 路由后，DNS 记录会自动创建。但有时需手动添加以确保即时生效。

5.6 SSL/TLS 设置

Cloudflare Dashboard → **SSL/TLS** → 设为 **Full（严格）**：

- 确保 SSL 证书
- 全程加密通信

六、关键差异详解

6.1 进程管理对比

功能	Armbian	Windows
自动重启	systemd `Restart=always`	无原生支持
开机自启	systemd `enable`	Windows Service
日志管理	journalctl	文件重定向
资源限制	systemd 配置	任务管理器

6.2 编码处理（Windows 特有）

Windows 默认 GBK 编码，会导致非 ASCII 字符输出乱码。项目代码已在 `main.py` 中做了三重编码保护：

```
# 1. 设置环境变量
os.environ['PYTHONIOENCODING'] = 'utf-8'
os.environ['PYTHONUTF8'] = '1'
```

```
# 2. 安全 print 函数
builtins.print = _safe_print  # 处理编码错误和管道断开

# 3. 重新配置 stdout/stderr
sys.stdout.reconfigure(encoding='utf-8', errors='replace')
```

6.3 进程池配置（Windows 特有）

```
# backend/app/api/__init__.py
# 自动检测平台并选择正确的多进程模式
import multiprocessing

mp_ctx_name = "spawn" if sys.platform == "win32" else "fork"
mp_ctx = multiprocessing.get_context(mp_ctx_name)
pool = ProcessPoolExecutor(
    max_workers=settings.MAX_WORKERS,
    mp_context=mp_ctx
)
```

- **Linux**：`fork` 模式（子进程继承父进程内存）
- **Windows**：`spawn` 模式（子进程重新导入模块，不支持 fork）

6.4 路径差异

路径	Armbian	Windows
项目根目录	`/opt/projects/NetworkPlanningTooV3`	`D:\mycode\NetworkPlanningTooV3`
后端目录	`backend/`	`backend\`
前端构建输出	`frontend/dist-renderer/`	`frontend\dist-renderer\`
数据目录	`data/`	`data\`
上传目录	`uploads/`	`uploads\`
日志目录	`backend/logs/`	`backend\logs\`

> 注意：项目中所有路径都使用 `pathlib.Path(__file__).parent` 来动态获取，**不依赖硬编码路径**，因此迁移到任何目录都可以正常工作。

七、Electron 桌面端（仅 Windows）

7.1 编译 TypeScript

```
cd frontend
npm run build:electron
```

编译产物在 `frontend/dist-electron/` 目录：

- `main.js` — Electron 主进程
- `preload.js` — 预加载脚本

7.2 启动桌面端

开发模式（Vite + Electron 同时启动）：

```
cd frontend
npm run dev
```

生产模式（FastAPI 已运行）：

```
# 方式1：脚本启动
.\start_electron_app.bat

# 方式2：手动
```

```
cd frontend
npx electron .
```

7.3 打包为 Windows 安装程序

```
# 安装 electron-builder（如未安装）
npm install --save-dev electron-builder

# 构建
npm run build

# 或直接使用配置
npx electron-builder --win --x64
```

> 注意：`electron-builder.json` 中已配置 NSIS 安装程序选项，打包后在 `frontend/dist/` 目录生成 `.exe` 安装包。

八、常见问题排查

8.1 后端无法启动

现象	原因	解决方案
`ModuleNotFoundError: No module named 'xxx	依赖缺失	安装缺失包：`pip install xxx`
`ImportError: Missing optional dependency	缺少 Excel 读取库	`pip install python-calamine`
`'str' object has no attribute 'get_start_	mp_context 传参错误	改为 `multiprocessing.get_context("spawn")`
`[Errno 10048]` 端口被占用	旧进程未退出	`taskkill /F /PID <PID>`
编码错误	Windows GBK 问题	设置 `PYTHONIOENCODING=utf-8` 环境变量

8.2 前端相关问题

现象	原因	解决方案
页面空白，控制台 404	前端未构建或路径错误	重新 `npm run build:vite`
API 请求 502	后端未启动	检查后端服务状态
CORS 错误	`.env` 未添加域名	在 `NPT_CORS_ORIGINS` 中添加
登录后跳转异常	JWT 密钥不一致	确认 `.env` 中 `NPT_JWT_SECRET_KEY` 一致

8.3 Tunnel 连接问题

现象	原因	解决方案
Error 1016 (Origin DNS error)	DNS 未配置或路由未同步	添加 CNAME 记录，等待 1-2 分钟
Error 1033	隧道未运行	启动 cloudflared 服务
访问进入错误机器	域名路由绑定到错误隧道	检查 Public Hostname 的隧道归属
`Cannot determine default origin certifica	未登录认证	运行 `cloudflared tunnel login`
`Failed to connect to new websocket`	网络问题或隧道断开	重启 cloudflared

8.4 迁移部署注意事项

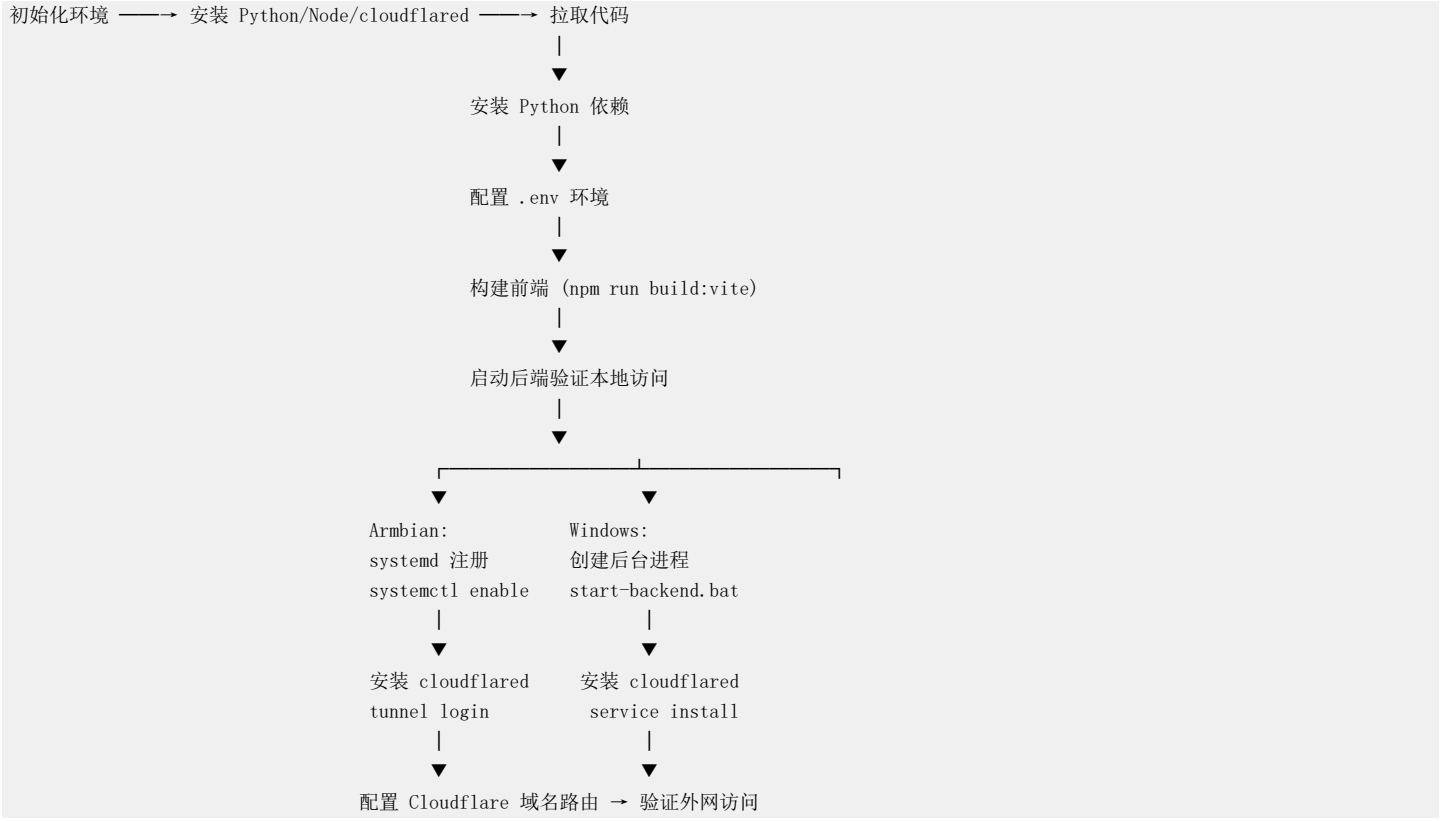
从一台机器迁移到另一台时，注意检查以下内容：

- [ ] Python 版本是否匹配（3.11+）
- [ ] Node.js 版本是否匹配（20+）
- [ ] 虚拟环境是否重建（venv 不跨平台/跨机器）

- [ ] `.env` 文件是否复制（含密钥）
- [ ] 数据结构目录是否存在（`data/`, `uploads/`, `outputs/`）
- [ ] CORS 配置是否更新为新域名
- [ ] Cloudflare Tunnel 是否需要创建新的隧道（不同机器建议不同隧道）
- [ ] 前端是否需要重新构建（平台无关，但建议重新构建）
- [ ] Windows 系统是否需要修改 `mp_context`（已在代码中自动处理）

九、部署清单

新机器部署完整流程



分平台速查

Armbian 部署速查：

```
# 1. 环境
sudo apt install -y python3 python3-pip python3-venv nodejs npm

# 2. 后端
python3 -m venv venv && source venv/bin/activate && pip install -r requirements.txt

# 3. 前端
cd frontend && npm install --legacy-peer-deps && npm run build:vite

# 4. 服务
sudo nano /etc/systemd/system/npt-backend.service    # 创建服务
sudo systemctl daemon-reload && sudo systemctl enable npt-backend && sudo systemctl start npt-b
sudo journalctl -u npt-backend -f                    # 查看日志

# 5. 隧道
cloudflared tunnel login                              # 认证
```

```
cloudflared tunnel create <隧道名>                    # 创建隧道
cloudflared tunnel route dns <隧道名> <域名>          # 配置 DNS
```

Windows 部署速查：

```
# 1. 后端
python -m venv venv
.\venv\Scripts\activate
pip install -r requirements.txt
pip install python-calamine

# 2. 前端
cd frontend
npm install --legacy-peer-deps
npm run build:vite

# 3. 启动后端
Start-Process -FilePath ".\venv\Scripts\python.exe" -ArgumentList "main.py" -WindowStyle Hidden

# 4. 验证
curl http://localhost:8000/health

# 5. 隧道（管理员 PowerShell）
cloudflared.exe service install <token>
net start cloudflared

# 6. 一键启动
.\scripts\start-all.bat
```

十、附录

附录 A：常用命令速查

操作	Armbian	Windows
启动后端	`sudo systemctl start npt-backend`	`.\venv\Scripts\python main.py`
停止后端	`sudo systemctl stop npt-backend`	`taskkill /F /PID <PID>`
重启后端	`sudo systemctl restart npt-backend`	停止后重新启动
查看后端日志	`journalctl -u npt-backend -f`	`type logs\backend.log`
启动隧道	`sudo systemctl start npt-tunnel`	`net start cloudflared`
停止隧道	`sudo systemctl stop npt-tunnel`	`net stop cloudflared`
查看隧道状态	`sudo systemctl status npt-tunnel`	`sc query cloudflared`

附录 B：目录结构

```
NetworkPlanningTooV3/
├── backend/                    # 后端 Python 项目
│   ├── main.py                 # 入口文件
│   ├── .env                    # 环境配置
│   ├── requirements.txt        # Python 依赖
│   ├── venv/                   # 虚拟环境（需创建）
│   ├── app/                    # 应用代码
│   │   ├── api/                # API 路由
│   │   │   ├── __init__.py     # 应用工厂 + ProcessPoolExecutor
│   │   │   └── v1/             # API v1 版本
│   │   ├── core/               # 核心配置
│   │   │   ├── config.py       # Settings 配置类
│   │   │   └── database.py     # 数据库
```

```
│   │   └── services/          # 业务逻辑
│   ├── logs/                  # 日志目录
│   ├── uploads/               # 上传文件
│   └── data/                  # 数据处理
├── frontend/                  # 前端 React + Electron
│   ├── package.json           # 依赖配置
│   ├── vite.config.ts         # 构建配置
│   ├── electron/              # Electron 桌面端
│   │   ├── main.ts            # 主进程
│   │   └── preload.ts         # 预加载
│   ├── dist-renderer/         # 构建产物
│   └── dist-electron/         # Electron 编译产物
├── scripts/                   # 部署脚本
│   ├── start-backend.bat      # Windows 后端启动
│   ├── start-tunnel.bat       # Windows 隧道启动
│   ├── start-all.bat          # Windows 一键启动
│   ├── build-backend.sh       # Linux 后端构建
│   └── build-frontend.sh      # Linux 前端构建
├── start_electron_app.bat     # Windows 桌面端启动
└── install_geopandas.bat      # geopandas 安装脚本
```

附录 C：项目配置说明

环境变量	说明	默认值
`NPT_HOST`	监听地址	`0.0.0.0`
`NPT_PORT`	监听端口	`8000`
`NPT_CORS_ORIGINS`	允许的跨域源	逗号分隔的 URL 列表
`NPT_JWT_SECRET_KEY`	JWT 签名密钥	生产环境必须修改
`NPT_MAX_WORKERS`	进程池 Worker 数	`max(1, cpu_count() - 1)`
`NPT_RELOAD`	开发热重载	`false`（生产）
`NPT_WORKERS`	uvicorn worker 数	`1`

> 本文档由项目迁移过程中的实践经验总结而成。如有更新，请以最新部署为准。